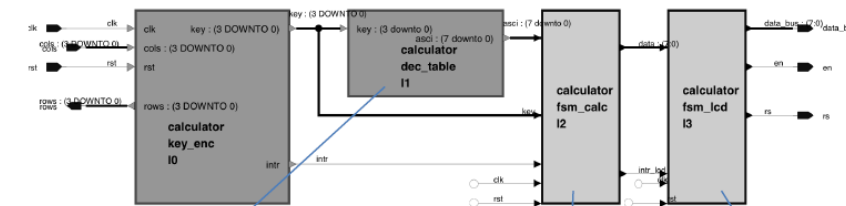
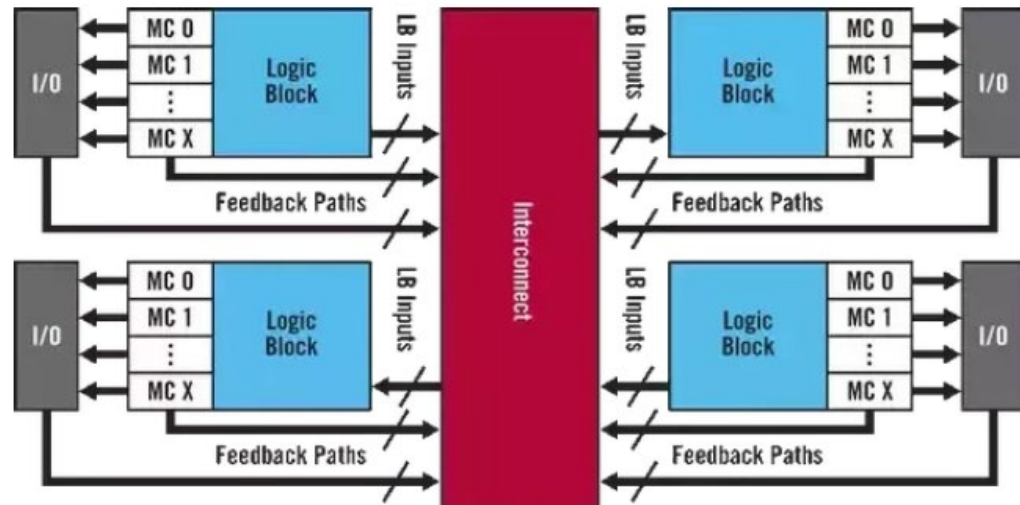


Lecture 9: UART, SPI and I2C

By: Emad Samuel Malki Ebeid

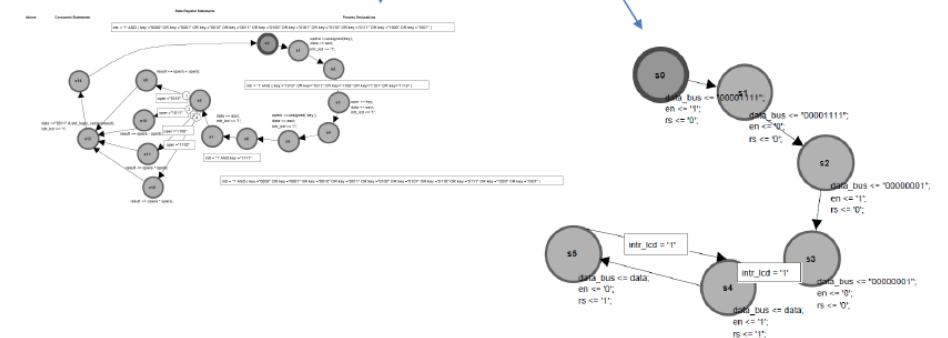
Summary of lecture 9



```

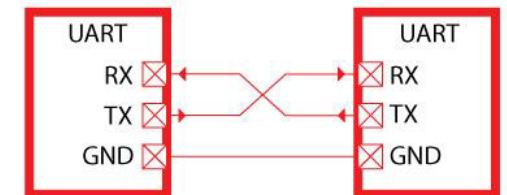
ENTITY dec_table IS
-- Declarations
port(key: in std_logic_vector(3 downto 0);
      ascii: out std_logic_vector(7 downto 0));
END dec_table ;

-- bds interface end
ARCHITECTURE rtl OF dec_table IS
BEGIN
process(key)
begin
if (key="0000") then ascii<="00110000";
elsif (key="0001") then ascii<="00110001";--0
elsif (key="0010") then ascii<="00110010";
elsif (key="0011") then ascii<="00110011";
elsif (key="0100") then ascii<="00110100";
elsif (key="0101") then ascii<="00110101";
elsif (key="0110") then ascii<="00110110";
elsif (key="0111") then ascii<="00110111";
elsif (key="1000") then ascii<="00111000";
elsif (key="1001") then ascii<="00111001";--9
elsif (key="1010") then ascii<="00110101";--A
elsif (key="1011") then ascii<="00110111";--B
elsif (key="1100") then ascii<="00110101";--C
elsif (key="1101") then ascii<="00110111";--D
elsif (key="1110") then ascii<="00110110";--E
else ascii<="00111101"; --F
end if;
end process;
END rtl;
    
```





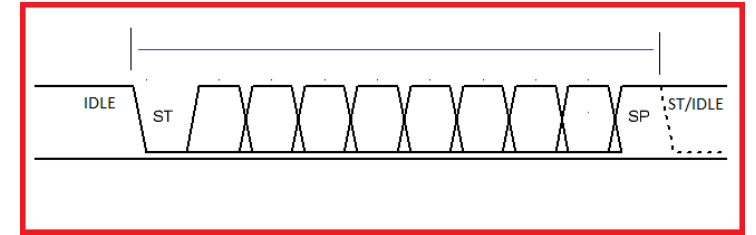
Serial interface: **Universal Asynchronous Receiver/Transmitter**



Serial communication introduction

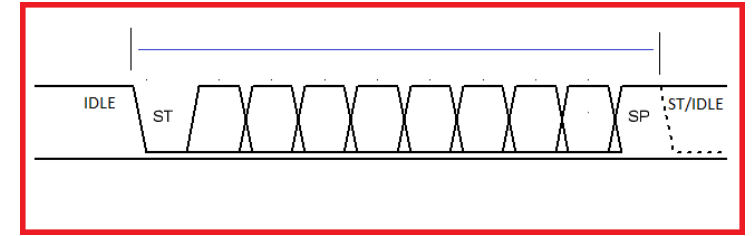
- Serial port is a serial communication physical interface through which information transfers in or out one bit at a time. Data transfer through serial ports connected the computer to devices such as terminals and various peripherals.
- Some computers, such as the IBM PC, used an integrated circuit called a UART, that converted characters to (and from) asynchronous serial form, and automatically looked after the timing and framing of data. A universal asynchronous receiver/transmitter (usually abbreviated UART) is a type of "asynchronous receiver/transmitter", a piece of computer hardware that translates data between parallel and serial forms

Baudrate



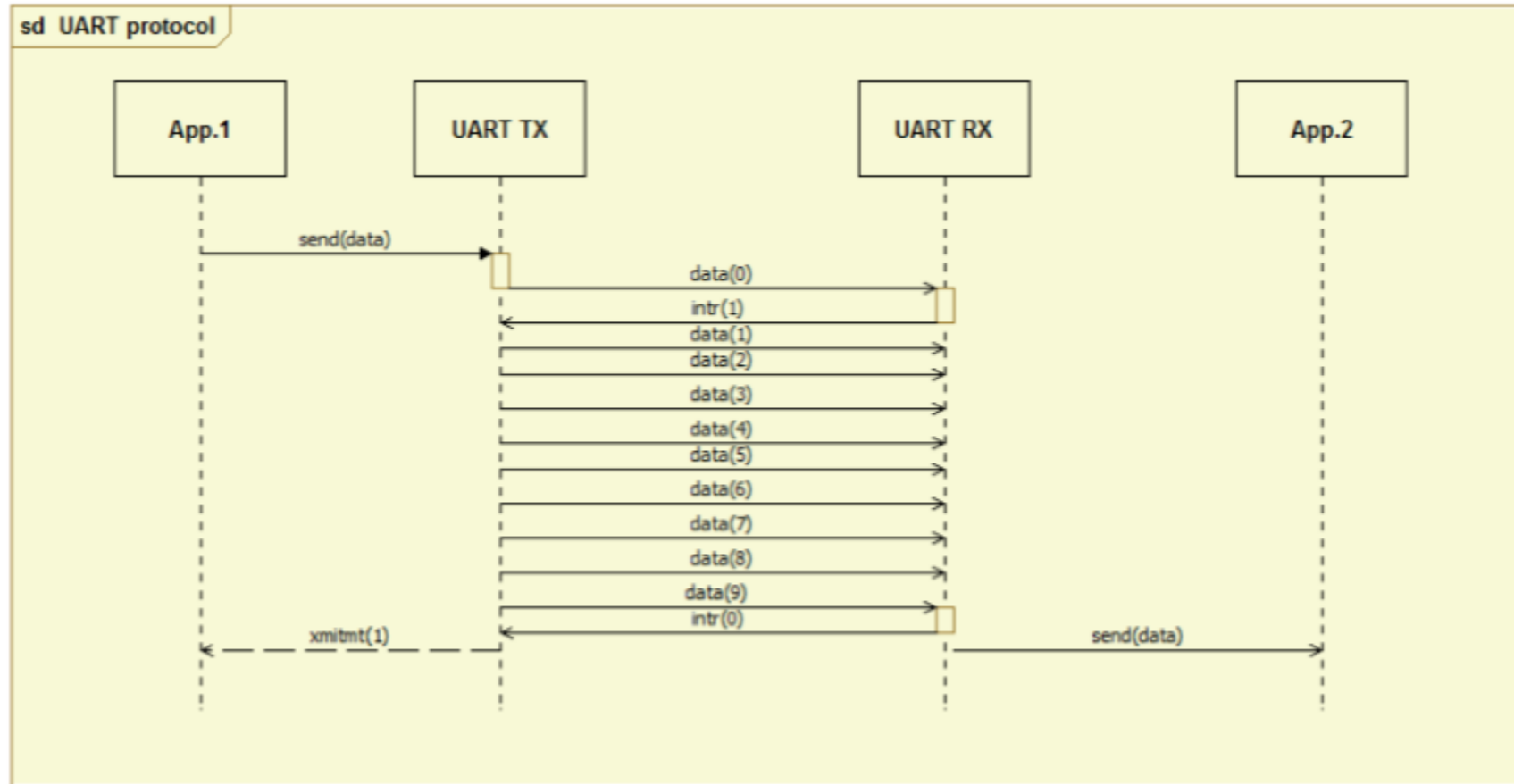
- **Baudrate:** In embedded designs, it is necessary to choose a proper oscillator to get the correct baud rate with little or no error. Some examples of common crystal frequencies and baud rates with no errors are: 300, 600, 1200, 1800, 2400, 4800, 7200, 9600, 14400, 19200, 38400, 57600, 115200 Bd
- **Data bits:** The number of data bits in each character can be 5 (for Baudot code), 6 (rarely used), 7 (for true ASCII), 8 (for any kind of data, as this matches the size of a byte), or 9 (rarely used). 8 data bits are almost universally used in newer applications. 5 or 7 bits generally only make sense with older equipment such as teleprinters.
- Most serial communications designs send the data bits within each byte LSB (Least Significant Bit) first. This standard is also referred to as "little-endian". Also, possible, but rarely used, is "big-endian" or MSB (Most Significant Bit) first serial communications. The order of bits is not usually configurable, but data can be byte-swapped only before sending.

Parity and stop bits

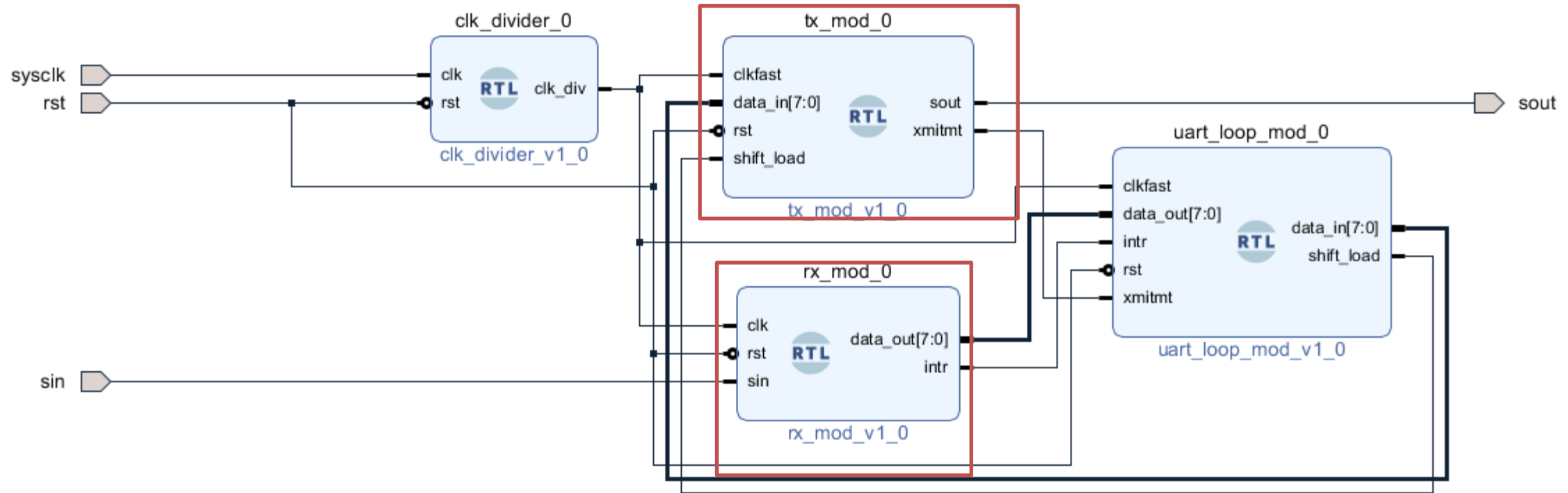


- **Parity:** Parity is a method of detecting errors in transmission. When parity is used with a serial port, an extra data bit is sent with each data character, arranged so that the number of 1 bits in each character, including the parity bit, is always odd or always even. If a byte is received with the wrong number of 1's, then it must have been corrupted. However, an even number of errors can pass the parity check.
- **Stop bits:** Stop bits sent at the end of every character allow the receiving signal hardware to detect the end of a character and to resynchronise with the character stream. Electronic devices usually use one stop bit. If slow electromechanical teleprinters are used, one-and-one half or two stop bits are required.

UART protocol



Overall system



- The code is in itsLearning

```

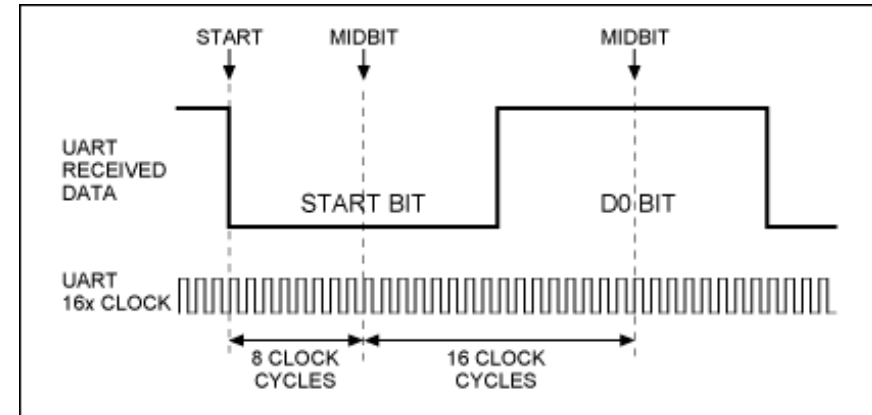
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_arith.all;

ENTITY rx_mod IS
    PORT(
        clk      : IN      std_logic;
        rst      : IN      std_logic;
        sin      : IN      std_logic;
        data_out  : OUT     std_logic_vector (7 DOWNTO 0);
        intr      : OUT     std_logic);
END rx_mod ;

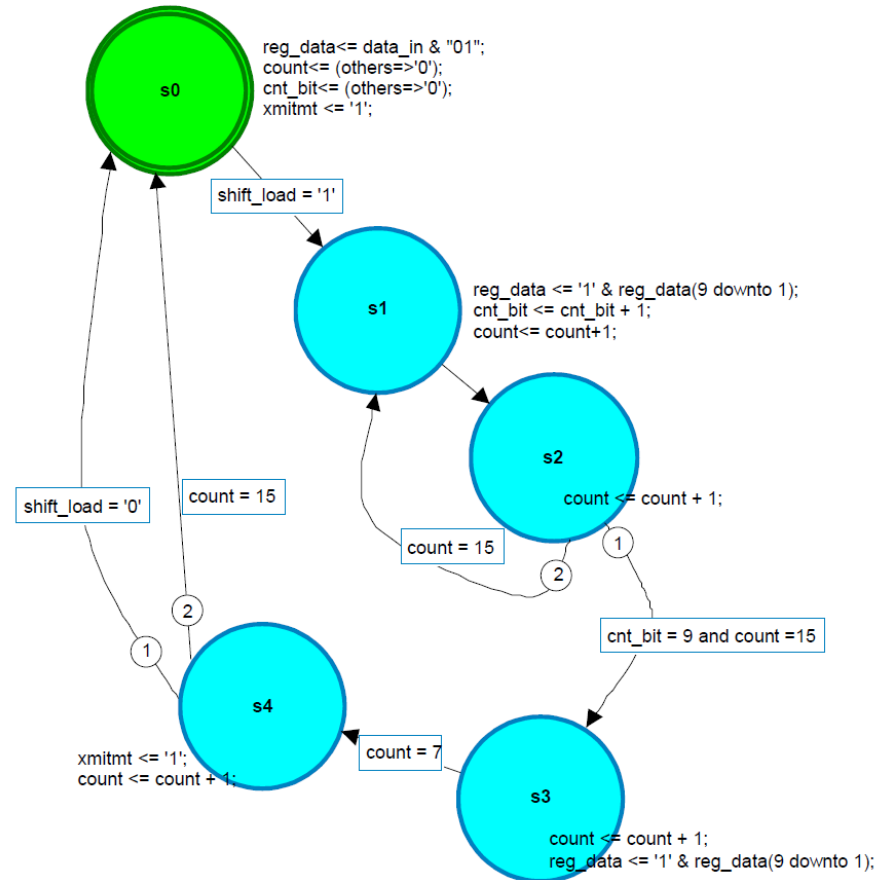
-- hds interface_end
ARCHITECTURE rtl OF rx_mod IS
    signal rxreg: std_logic_vector(9 downto 0);
    signal count: unsigned (3 downto 0);
    signal rxmt: std_logic;
    signal rxin,start_flag: std_logic;
    begin
        process (clk, rst)
        begin
            if (rst = '0') then
                count <= (others => '0');
                rxmt <= '1';
                rxreg <= (others => '1');
                intr <= '0';
                rxin <= '1';
                start_flag<='0';
            elsif (rising_edge(clk)) then
                rxin<=sin;
                if (rxmt = '1' and rxin = '0') then
                    count <= (others => '0');
                    rxmt <= '0';
                    rxreg <= (others => '1');
                    start_flag<='0';
                elsif (count = 7 and rxmt = '0' and rxin = '0' and start_flag='0') then
                    rxreg <= rxin & rxreg(9 downto 1);
                    count <= (others => '0');
                    start_flag<='1';
                elsif (count = 15 and rxmt = '0') then
                    rxreg <= rxin & rxreg(9 downto 1);
                    count <= count + 1;
                else
                    count <= count + 1;
                end if;
                if (rxmt = '0' and rxreg(9) = '1' and rxreg(0) = '0') then
                    intr <= '1';
                    rxmt <= '1';
                else
                    intr <= '0';
                end if;
            end if;
        end process;
        data_out <= rxreg(8 downto 1);
    END rtl;

```

Receiver



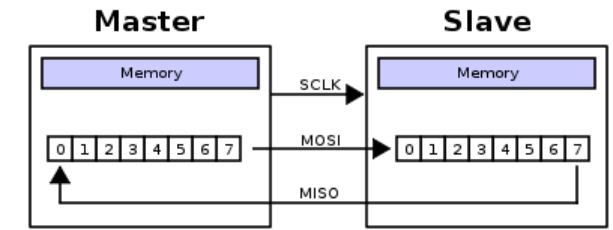
Transmitter



- Default values are 0

Serial Peripheral Interface (SPI)

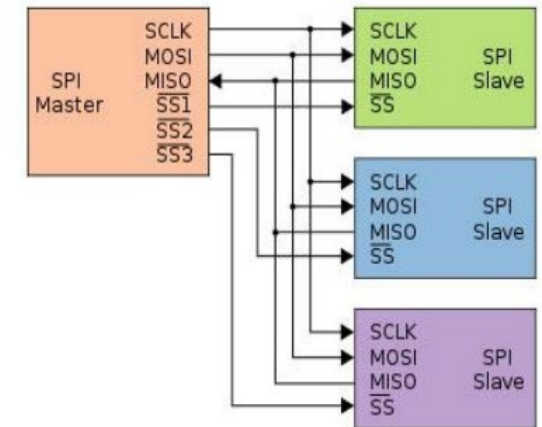
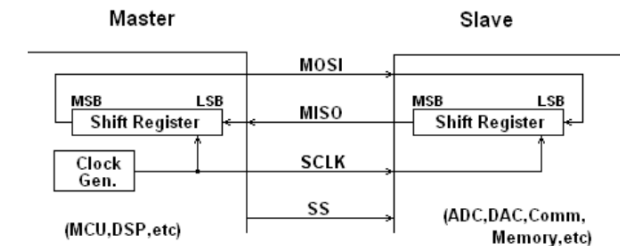
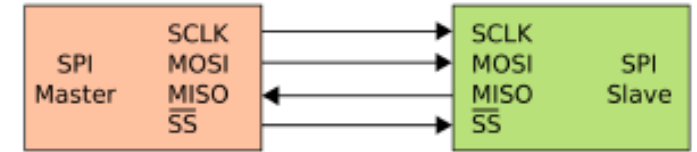
SPI



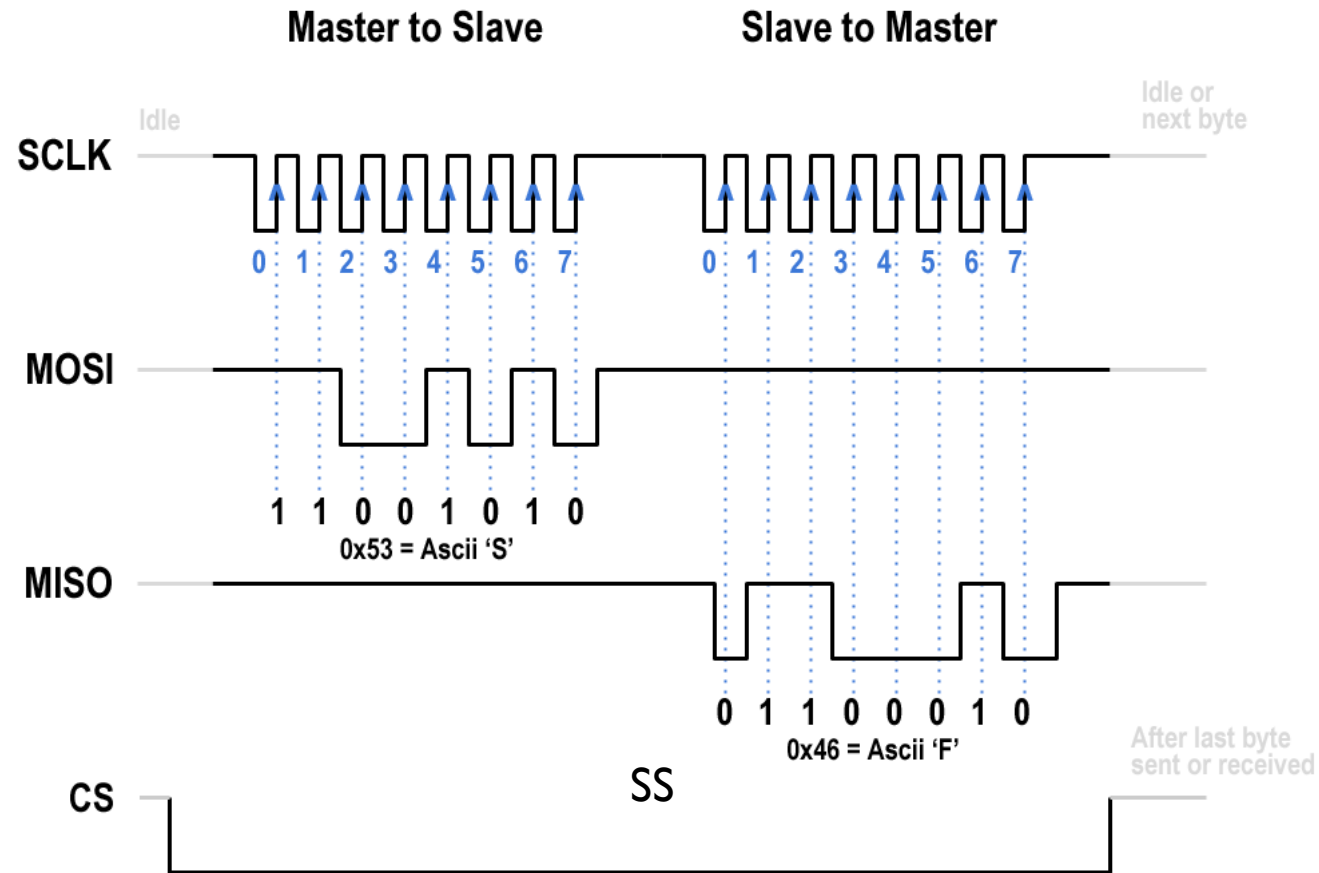
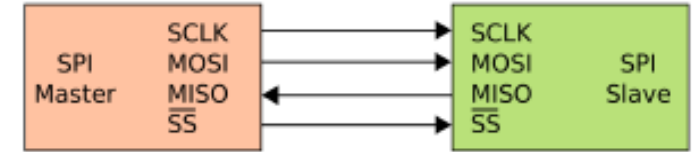
- Serial Peripheral Interface (SPI) is a synchronous serial communication interface for short-distance communication.
- Full duplex mode using a master-slave architecture with a single master.
- SPI communication was used to connect devices such as printers, cameras, scanners, etc. to a desktop computer; nowadays it is replaced by USB.
- In robotics, SPI is still utilized for connecting sensors, actuators, etc.
- SPI runs using a master/slave set-up and can run in full duplex mode (i.e., signals can be transmitted between the master and the slave simultaneously).
- When multiple slaves are present, SPI requires no addressing to differentiate between these slaves.
- There is no standard communication protocol for SPI.

SPI

- SPI works by shifting bits between two registers of the same word size. Data is exchanged through the two connections, MOSI and MISO.
- One bit is shifted for each cycle of the serial clock.
- This yields a constant transfer rate. The first bit delivered is the most significant bit and the least significant bit is the last.
 - SCLK is the serial clock.
 - MOSI is the output from the master to the slave.
 - MISO is the output from the slave to the master.
 - SS is the slave select, which is used to wake up the slave.



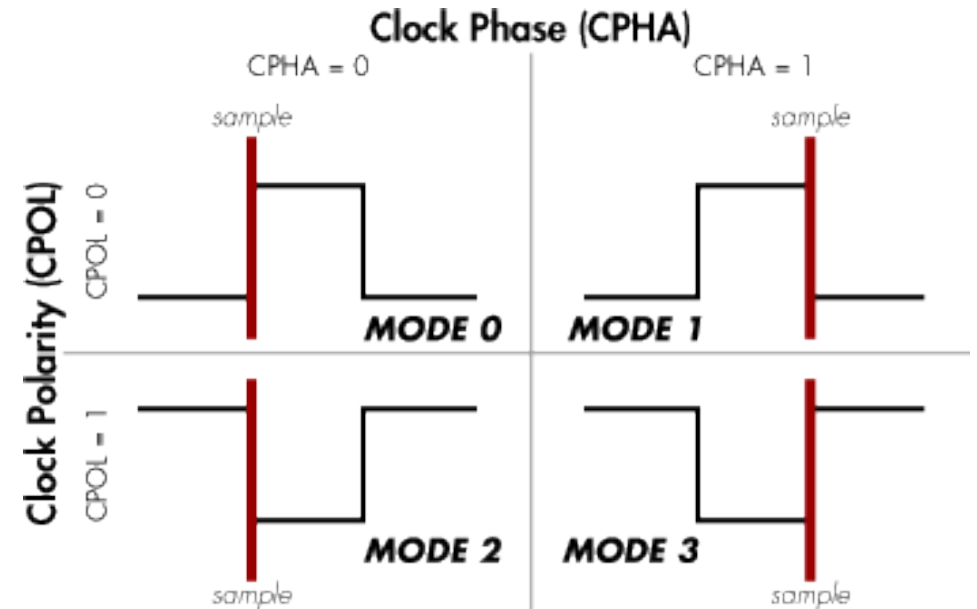
How it works



SPI Modes

- It aims at adding more flexibility to the communication channel between the master and slave.
- Its a combination of polarity and phases (CPOL, CPHA)
 - CPOL (**C**lock **POL**arity) and CPHA (**C**lock **PHA**se)

Mode	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

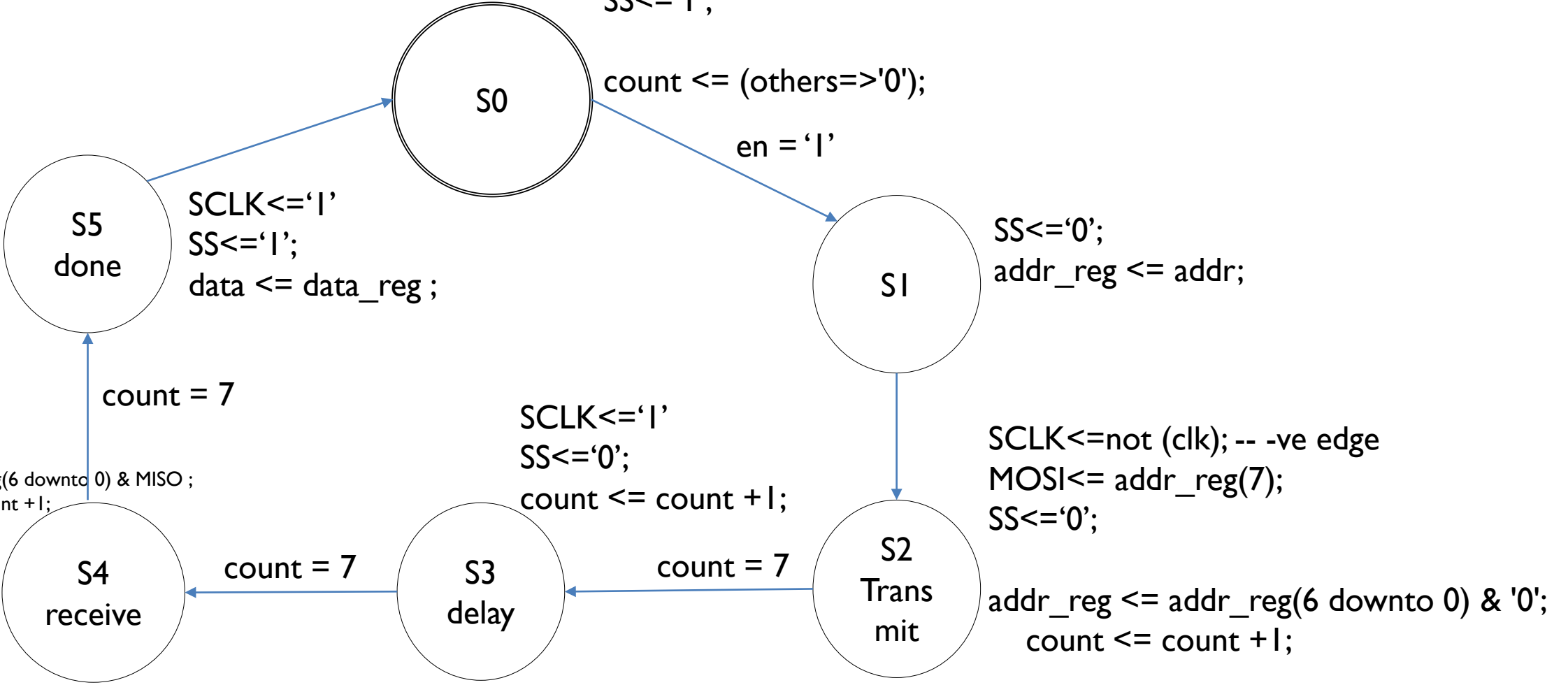
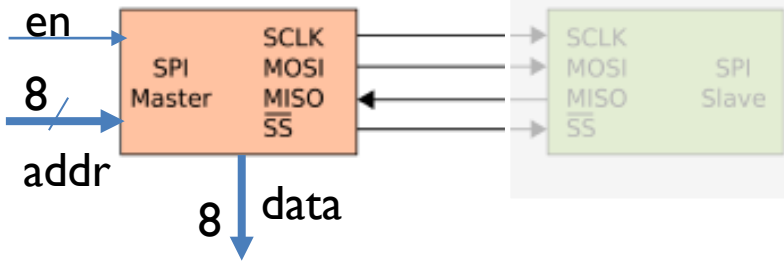


Internal signals:

```
addr_reg, data_reg: std_logic_vector(7 downto 0) ;  
count: unsigned(2 downto 0) ;
```

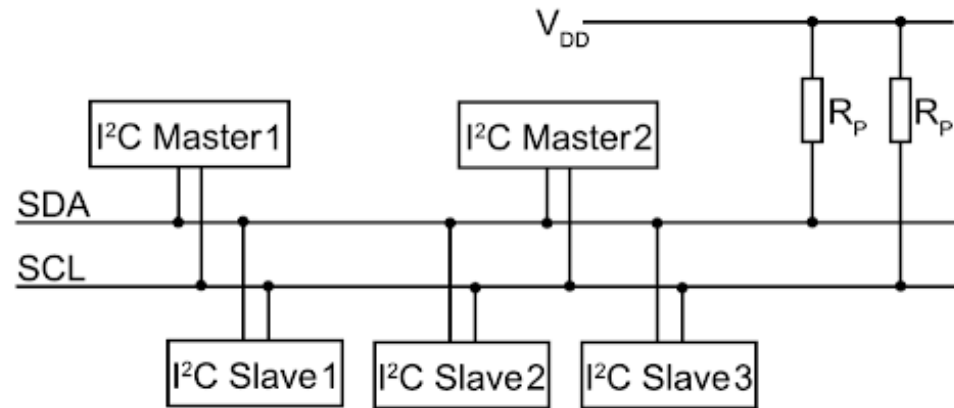
SPI FSM: Master

```
SCLK<='0';  
MOSI<='0';  
SS<='1';
```



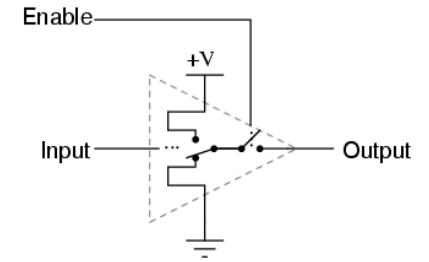
Assignment 3

I²C

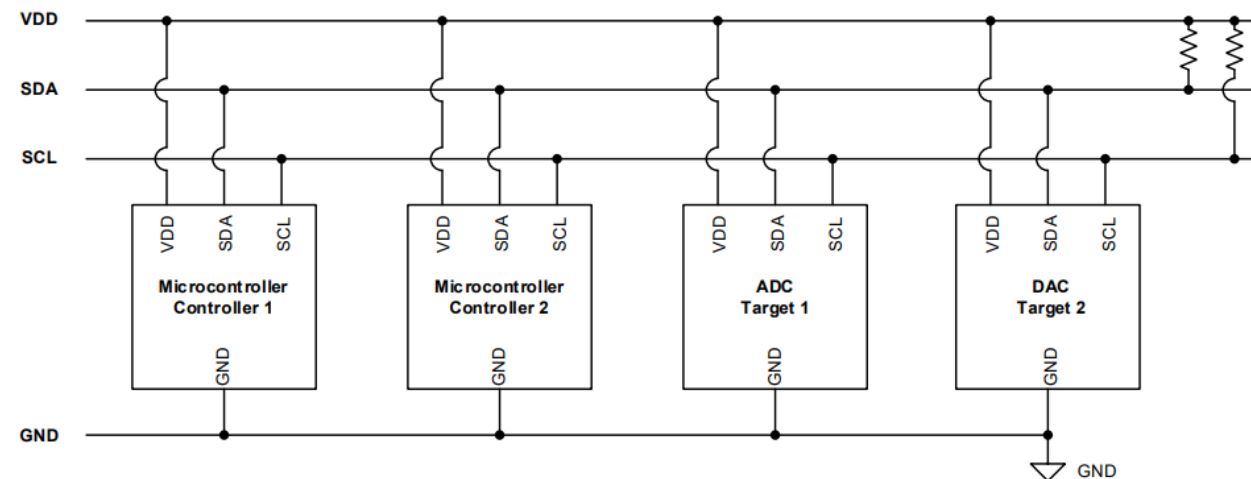


Reference: <https://training.ti.com/sites/default/files/docs/adcs-i2c-introduction-the-protocol-presentation.pdf>

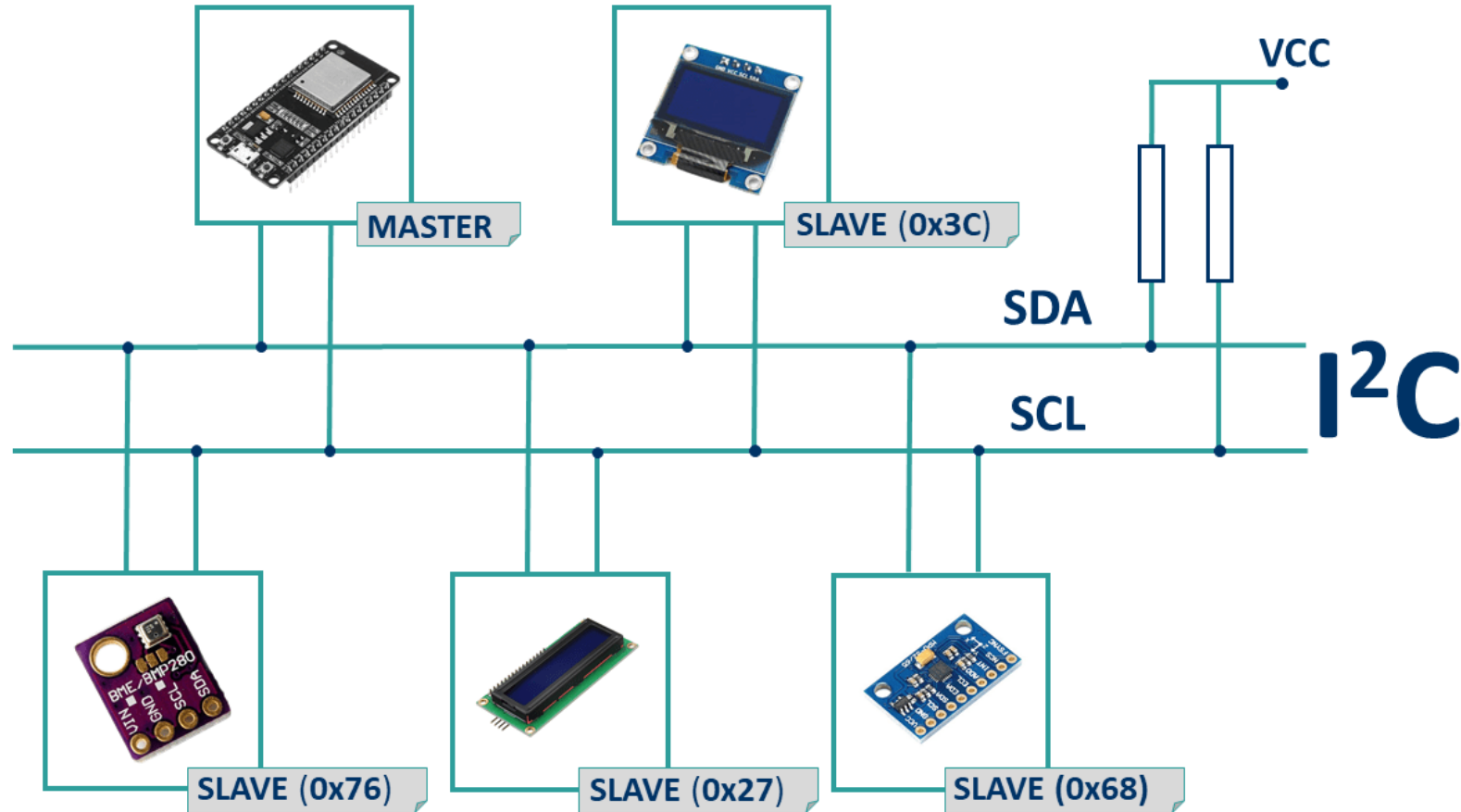
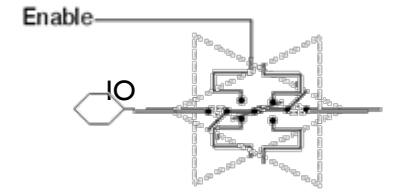
Inter-IC (I2C)



- It is a serial synchronous bus invited by Philips Semiconductor in 1982.
- It is widely used to connect low-speed ICs to processing units in short-distance, intra-board communication. Common speeds are 100kbps standard mode and 400kbps fast mode.
- Only two communication lines for all devices on the bus; Serial Data Line (SDA) and Serial Clock Line (SCL) pulled up with resistors.



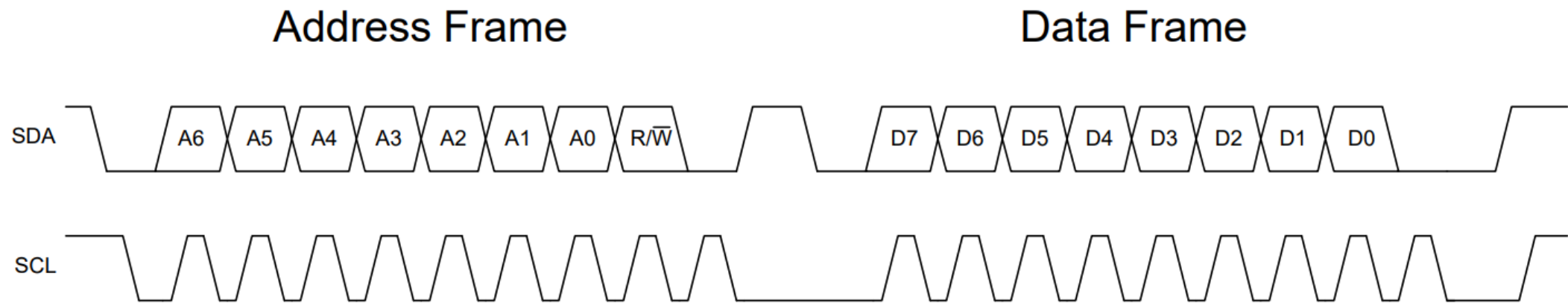
Example

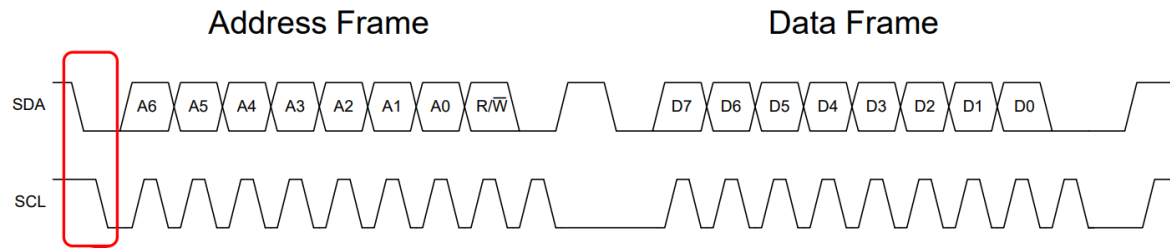


Protocol

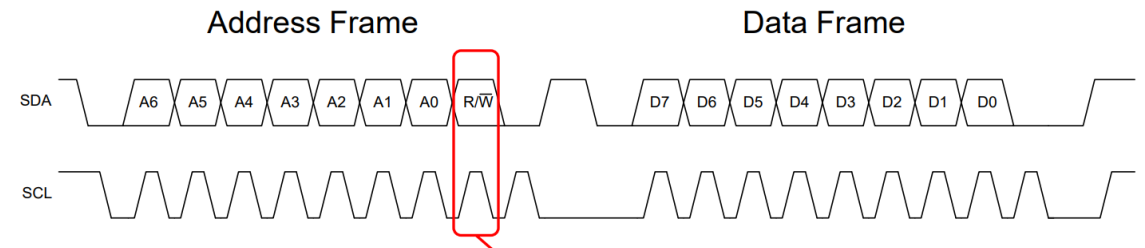
- Start Condition:
 - The master node leaves the SCL high and pulls SDA low.
 - This puts all slave devices on notice that a transmission is about to start.
 - If two masters want to transmit at the same time, the master who owns the bus wins.
 - The address frame is always first with the MSB is sent first followed by R(1)/W(0).
 - NACK/ACK bit, it is the 9th bit in the frame. Once the 8 bits are sent, the receiving device is given control over SDA.
 - If the receiving device did not pull the SDA line low, then the exchange halts, and the master decide how to proceed.
 - After the address is received, the master will continue generating the clock pulses at a regular interval.
 - When all data are sent, the master sends stop bits.

Timing Diagram

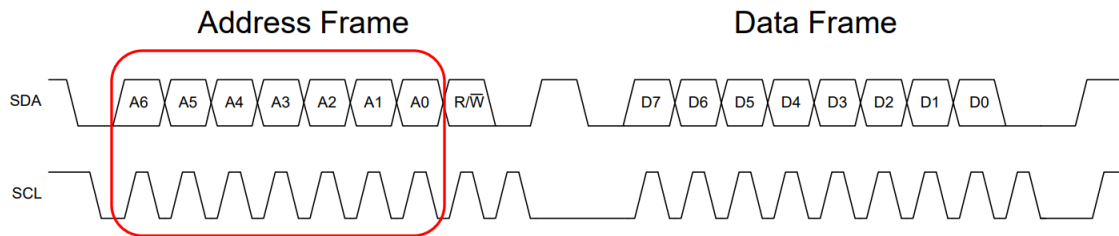




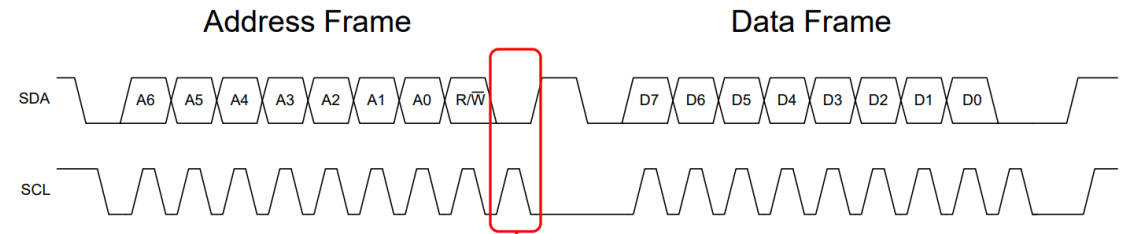
An I2C START condition comes from the controller and sends SDA low before SCL is sent low to claim the bus



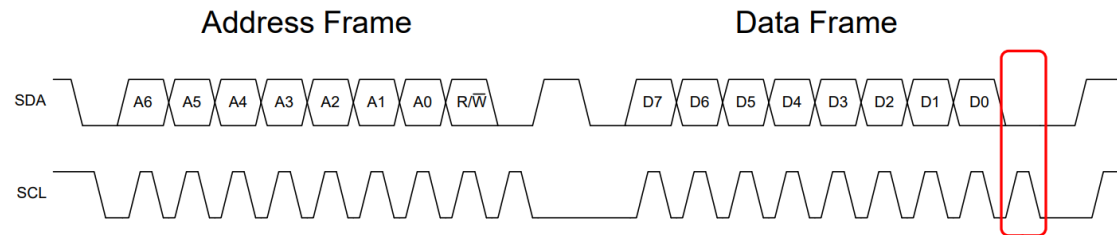
R/W bit indicates the direction of communication
 1: Controller wants to read from the target device
 0: Controller wants to write to the target device



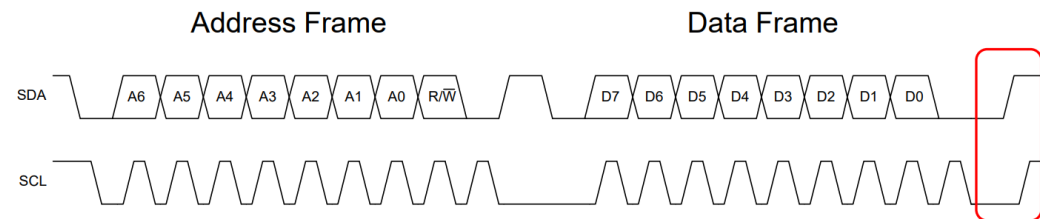
Seven bits make up the I2C address



SDA is pulled down as an ACKT (acknowledge, target)
 After the address byte, the target device ACKs the communication



ACK follows each data frame
 Write to the target – ACKT comes from the target device
 Read from the target – ACKC comes from the controller device



I2C STOP condition comes from the controller and sends
 SDA high before SCL is sent high to release the bus

